

Lab Report for Information Security

Lab 1: Implementation of Caesar Cipher

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 2: Implementation of Vigenere Cipher

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 3: Implementation of Playfair Cipher

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 4: Implementation of Rail Fence Ciphers

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 5: Implementation of password based authentication system

- i) Introduction/Concept
- ii) Program code
- iii) Output

[Note: You have already implemented in your project]

Caesar Cipher implementation

```
#include<stdio.h>
#include<conio.h>
int main()
{
    char message[100], ch;
    int i, key;
    clrscr();
    printf("Enter a message to encrypt: ");
    gets(message);
    printf("Enter key: ");
    scanf("%d", &key);
    for(i = 0; message[i] != '\0'; ++i)
    {
        ch = message[i];
        if(ch >= 'a' && ch <= 'z')
        {
```

```

        ch = ch + key;
        if(ch > 'z')
        {
            ch = ch - 'z' + 'a' - 1;
        }
        message[i] = ch;
    }
    else if(ch >= 'A' && ch <= 'Z')
    {
        ch = ch + key;
        if(ch > 'Z')
        {
            ch = ch - 'Z' + 'A' - 1;
        }
        message[i] = ch;
    }
}
printf("Encrypted message: %s", message);
getch();
return (0);
}

```

Implementation of Vigenere Cipher

```

#include<stdio.h>
#include<string.h>
#include<conio.h>

int main()
{
    char msg[200],key[200];
    int msgLen,keyLen,i,j;
    char newKey[200],encryptedMsg[200],decryptedMsg[200];
    clrscr();
    printf("Input message:");
    gets(msg);
    printf("Input key:");
    gets(key);
    msgLen=strlen(msg);
    keyLen=strlen(key);
    //generating new key
    for(i=0,j=0;i<msgLen;++i,++j)
    {
        if(j==keyLen)
            j=0;
        newKey[i]=key[j];
    }
    newKey[i] = '\0';
    //encryption
    for(i=0;i<msgLen;++i)

```

```

        encryptedMsg[i]=((msg[i]+newKey[i])%26)+'A';
encryptedMsg[i]='\0';
//decryption
for(i=0;i<msgLen;++i)
    decryptedMsg[i]=(((encryptedMsg[i]-newKey[i])+26)%26)+'A';
decryptedMsg[i]='\0';
printf("Original Message:%s",msg);
printf("\nKey:%s",key);
printf("\nNew Generated Key: %s",newKey);
printf("\nEncrypted Message: %s",encryptedMsg);
printf("\nDecrypted Message: %s",decryptedMsg);
getch();
return 0;
}

```

Implementation of Playfair Cipher

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h>

#define SIZE 30

// Function to convert the string to lowercase
void toLowerCase(char plain[], int ps)
{
    int i;
    for (i = 0; i < ps; i++) {
        if (plain[i] > 64 && plain[i] < 91)
            plain[i] += 32;
    }
}

// Function to remove all spaces in a string
int removeSpaces(char* plain, int ps)
{
    int i, count = 0;
    for (i = 0; i < ps; i++)
        if (plain[i] != ' ')
            plain[count++] = plain[i];
    plain[count] = '\0';
    return count;
}

// Function to generate the 5x5 key square
void generateKeyTable(char key[], int ks, char keyT[5][5])
{
    int i, j, k, flag = 0, *dicty;

```

```

// a 26 character hashmap
// to store count of the alphabet
dicty = (int*)calloc(26, sizeof(int));
for (i = 0; i < ks; i++) {
    if (key[i] != 'j')
        dicty[key[i] - 97] = 2;
}

dicty['j' - 97] = 1;

i = 0;
j = 0;

for (k = 0; k < ks; k++) {
    if (dicty[key[k] - 97] == 2) {
        dicty[key[k] - 97] -= 1;
        keyT[i][j] = key[k];
j++;
        if (j == 5) {
            i++;
            j = 0;
        }
    }
}

for (k = 0; k < 26; k++) {
    if (dicty[k] == 0) {
        keyT[i][j] = (char)(k + 97);
        j++;
        if (j == 5) {
            i++;
            j = 0;
        }
    }
}

// Function to search for the characters of a digraph
// in the key square and return their position
void search(char keyT[5][5], char a, char b, int arr[])
{
    int i, j;

    if (a == 'j')
        a = 'i';
    else if (b == 'j')
        b = 'i';

    for (i = 0; i < 5; i++) {

```

```

        for (j = 0; j < 5; j++) {

            if (keyT[i][j] == a) {
                arr[0] = i;
                arr[1] = j;
            }

            else if (keyT[i][j] == b) {
                arr[2] = i;
                arr[3] = j;
            }
        }
    }
}

```

```

// Function to find the modulus with 5
int mod5(int a) { return (a % 5); }

```

```

// Function to make the plain text length to be even
int prepare(char str[], int ptrs)
{
    if (ptrs % 2 != 0) {
        str[ptrs++] = 'z';
        str[ptrs] = '\0';
    }
    return ptrs;
}

```

```

// Function for performing the encryption
void encrypt(char str[], char keyT[5][5], int ps)
{
    int i, a[4];

    for (i = 0; i < ps; i += 2) {

        search(keyT, str[i], str[i + 1], a);

        if (a[0] == a[2]) {
            str[i] = keyT[a[0]][mod5(a[1] + 1)];
            str[i + 1] = keyT[a[0]][mod5(a[3] + 1)];
        }
        else if (a[1] == a[3]) {
            str[i] = keyT[mod5(a[0] + 1)][a[1]];
            str[i + 1] = keyT[mod5(a[2] + 1)][a[1]];
        }
        else {
            str[i] = keyT[a[0]][a[3]];
            str[i + 1] = keyT[a[2]][a[1]];
        }
    }
}

```

```

// Function to encrypt using Playfair Cipher
void encryptByPlayfairCipher(char str[], char key[])
{
    char ps, ks, keyT[5][5];
    // Key
    ks = strlen(key);
    ks = removeSpaces(key, ks);
    toLowerCase(key, ks);
    // strlwr(key);
    // Plaintext
    ps = strlen(str);
    toLowerCase(str, ps);
    ps = removeSpaces(str, ps);

    ps = prepare(str, ps);

    generateKeyTable(key, ks, keyT);

    encrypt(str, keyT, ps);
}

int main()
{
    char str[SIZE], key[SIZE];
    clrscr();
    // Key to be encrypted
    printf("Input the key text:");
    gets(key);
    // Plaintext to be encrypted
    printf("Input the plain text:");
    gets(str);
    // encrypt using Playfair Cipher
    encryptByPlayfairCipher(str, key);
    printf("Cipher text: %s\n", str);
    getch();
    return 0;
}

```

Rail Fence Ciphers

```

#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    int i,j,k,l;
    char a[20],c[20],d[20];
    clrscr();
    printf("\n\nEnter the input string : ");

```

```

gets(a);
l=strlen(a);
/*Encrypting*/
for(i=0,j=0;i<l;i++)
{
    if(i%2==0)
        c[j++]=a[i];
}
for(i=0;i<l;i++)
{
    if(i%2==1)
        c[j++]=a[i];
}
c[j]='\0';
printf("\nEncrypted text:");
printf("%s",c);
/*Decrypting*/
if(l%2==0)
    k=l/2;
else
    k=(l/2)+1;
for(i=0,j=0;i<k;i++)
{
    d[j]=c[i];
    j=j+2;
}
for(i=k,j=1;i<l;i++)
{
    d[j]=c[i];
    j=j+2;
}
d[l]='\0';
printf("\nDecryption Text: ");
printf("%s",d);
getch();
}

```